

SECURITY ASSESSMENT REPORT

Vulnerable Docker 1

Abstract

The findings shows the danger of outdated CMS versions and misconfigured container security. Key stages included User Enumeration, Credential Exploitation, and OS Command Injection. The attack culminated in a Docker Escape via an insecure socket, resulting in full Root access to the host machine

Prepared By:

Name : Ravipudi Yoganandh

Enrollment No:2203031260195

Roll No: 125

Date : 14 Mar 2026

Table of Contents

- 1. Introduction
- 2. Scope of Testing
- 3. Testing Methodology
- 4. Tools Used
- 5. Target Application Information
- 6. Testing Environment Setup
- 7. Reconnaissance
- 8. Vulnerability Findings
 - Vuln-01- **Information Disclosure**
 -
- 9. Conclusion
- 10. References

1. Introduction

The objective of this engagement was to perform a comprehensive security assessment on the **Vulnerable Docker: 1** environment. This assessment focused on identifying exploitable vulnerabilities, security misconfigurations, and potential lateral movement paths within the containerized architecture. Docker environments are widely used in modern application deployment, and misconfigurations within containers or the host system can lead to serious security risks if not properly managed.

During this assessment, various penetration testing techniques were applied to simulate the actions of a real-world attacker. The testing process included reconnaissance, service enumeration, vulnerability identification, exploitation attempts, and privilege escalation where possible. Special attention was given to weaknesses related to container isolation, exposed services, insecure permissions, and improper configuration of Docker components.

The primary goal of this assessment was to evaluate the overall security posture of the environment and determine whether an attacker could gain unauthorized access to the system or escalate privileges within the container or host machine. By identifying these weaknesses, the assessment aims to highlight security gaps and provide actionable remediation recommendations to help strengthen the environment, reduce the attack surface, and improve the overall resilience of the system against potential cyber threats.

1. Scope of Testing

The scope of this assessment was strictly limited to the following assets and methodologies:

- Target Asset: Vulnerable Docker: 1 Virtual Machine.
- Target IP Address: 192.168.42.6 (identified via ARP discovery).
- Included Services: All open TCP/UDP ports discovered during the reconnaissance phase, specifically focusing on the web application (Port 8000) and SSH (Port 22).
- Exclusions: No Denial-of-Service (DoS) attacks were performed to ensure environment stability. Testing was confined to the local virtualized subnet 192.168.42.0/24

2. Testing Methodology

- Reconnaissance: Active scanning to identify live hosts and open ports.
- Enumeration: Deep-diving into services (WordPress versioning, directory discovery).
- Exploitation: Attempting to gain initial access via known vulnerabilities (CVEs).
- Post-Exploitation: Escalating privileges and attempting to escape the Docker container to the host system.

6. Information Gathering & Reconnaissance

- **Step 1:** Perform an aggressive **Nmap** scan (`nmap -sV -sC -p- 192.168.42.6`) to identify all open ports and service versions.

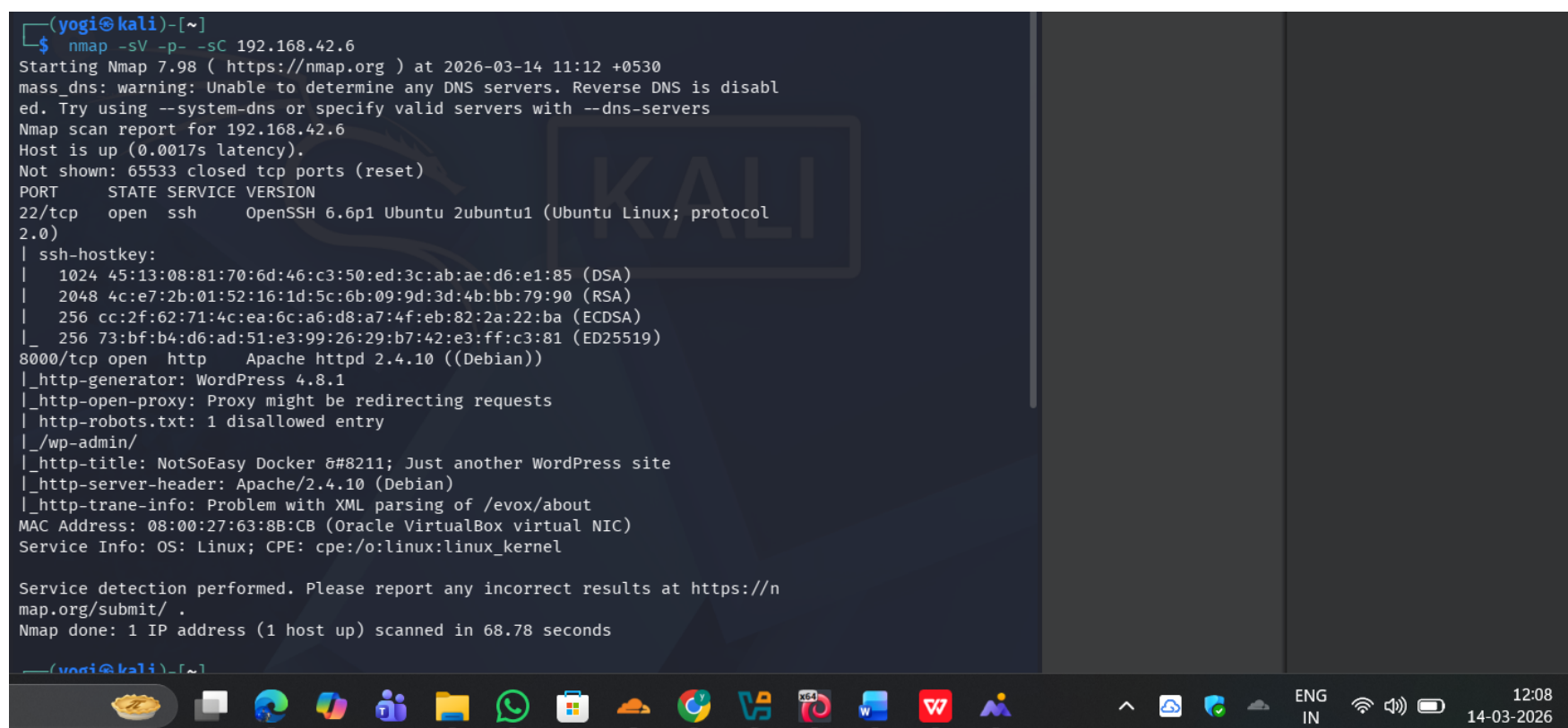
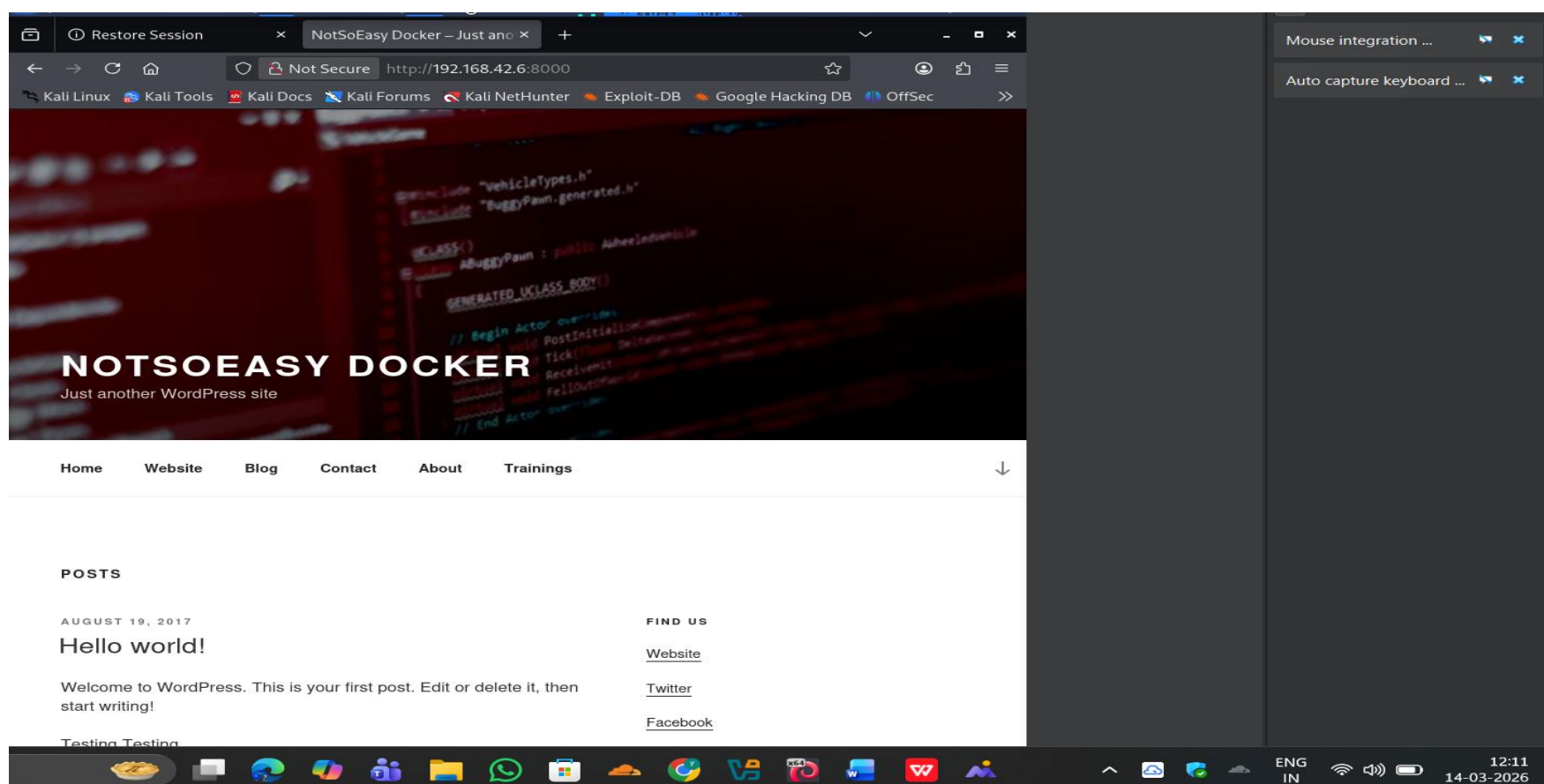


Image 2

- **Step 2:** Access the web application via a browser at <http://192.168.42.6:8000> to verify the site content and look for entry points.



- **Step 5:** Run **Gobuster** to discover hidden directories like `/wp-admin/`, `/secret/`, or configuration backups.

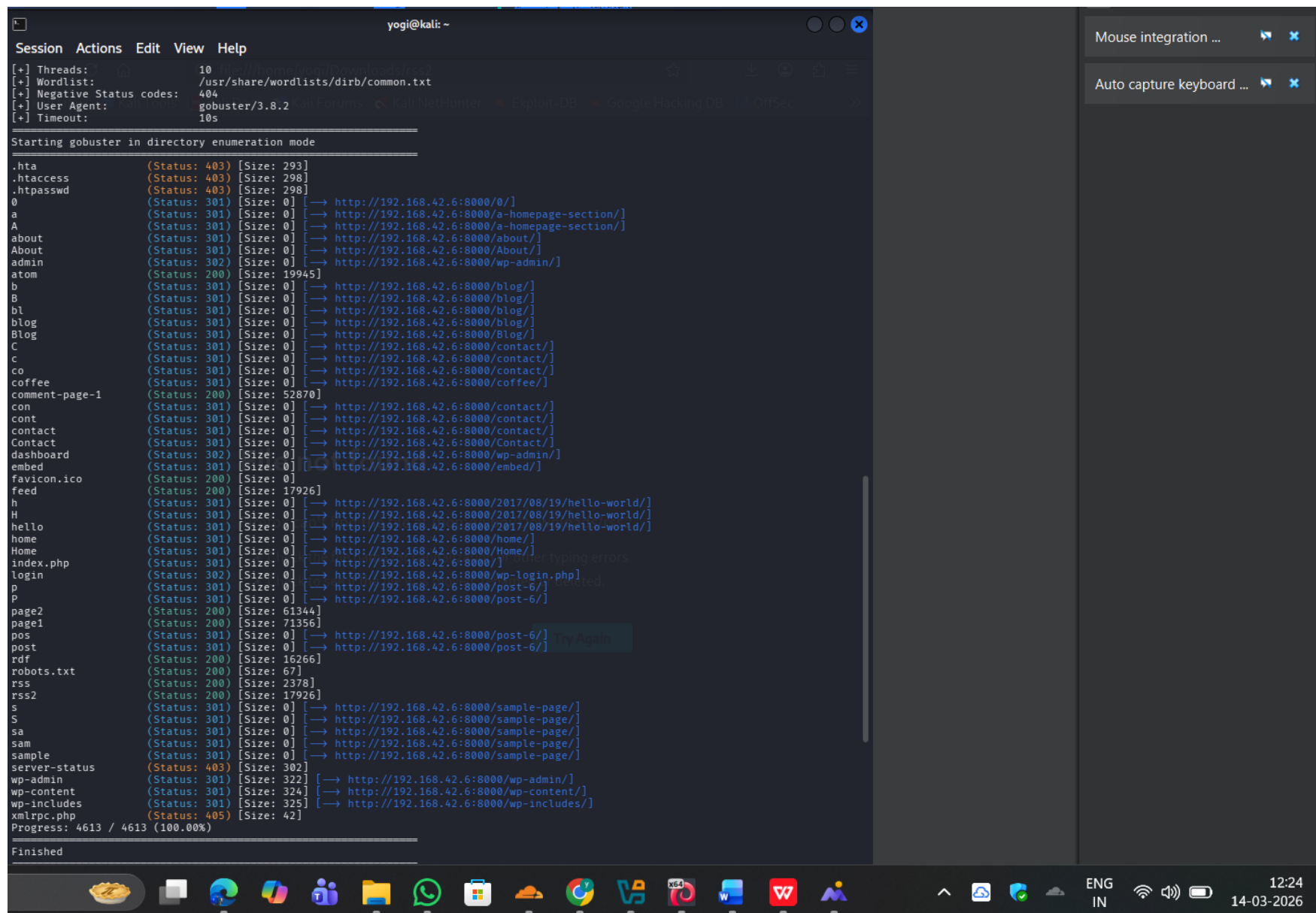


Image 3

- **Step 4:** Execute a **WPScan** (`wpscan --url http://192.168.42.6:8000 --enumerate-u`) to find valid usernames for potential brute-forcing.

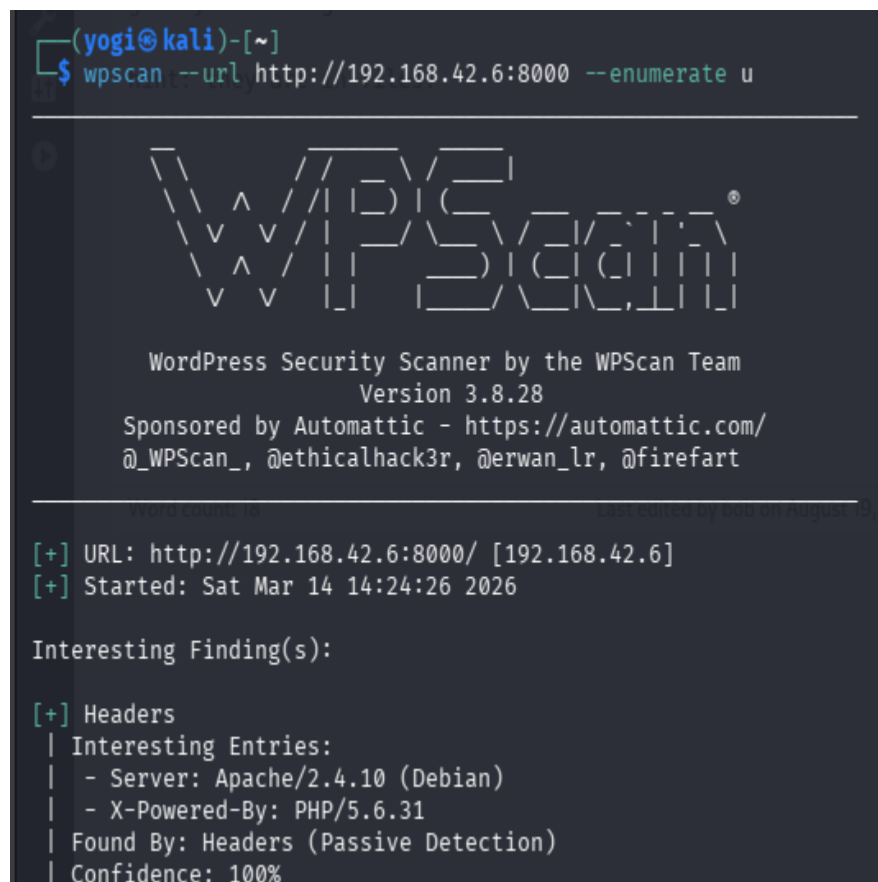


Image 4

```
Brute Forcing Author IDs - Time: 00:00:00 (10 / 10)

[i] User(s) Identified:

[+] bob
| Found By: Author Posts - Author Pattern (Passive Detection)
| Confirmed By:
| Rss Generator (Passive Detection)
| Wp Json Api (Aggressive Detection)
| - http://192.168.42.6:8000/wp-json/wp/v2/users/?per_page=100&page=1
| Rss Generator (Aggressive Detection)
| Author Id Brute Forcing - Author Pattern (Aggressive Detection)
| Login Error Messages (Aggressive Detection)

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.co

[+] Finished: Sat Mar 14 14:24:29 2026
[+] Requests Done: 31
```

Image 5

- **Finding:** confirming the username is **bob**.

After confirming through my initial reconnaissance that the target is running a vulnerable version of **WordPress (4.8.1)** and identifying a valid system user named '**bob**', I am now moving into the **Exploitation Phase**.

The command shown on the “image 6” is a targeted **Dictionary Attack**. Since the automated scan identified 'Bob' as a high-value target, I am using **WPScan** to systematically test every entry in the **RockYou.txt** wordlist against his account.

```
(yogi@kali)~$ wpscan --url http://192.168.42.6:8000 \
-U bob \
-P/home/yogi/Downloads/rockyou.txt

WordPress
WordPress Security Scanner by the WPScan Team
Version 3.8.28
Sponsored by Automattic - https://automattic.com/
@_WPScan_, @ethicalhack3r, @erwan_lr, @firefart

[+] URL: http://192.168.42.6:8000/ [192.168.42.6]
[+] Started: Sat Mar 14 14:35:08 2026

Interesting Finding(s):
[+] Headers
| Interesting Entries:
```

Image 6

```
Trying bob / Liverpool Time: 00:00:02 (40 / 143432) 0.00% ETA: 11:11:11
[!] Valid Combinations Found:
| Username: bob, Password: Welcome1

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.co
m/register

[+] Finished: Sat Mar 14 15:06:56 2026
[+] Requests Done: 213
[+] Cached Requests: 7
[+] Data Sent: 66.425 KB
[+] Data Received: 506.166 KB
[+] Memory used: 283.559 MB
[+] Elapsed time: 00:00:15

(yogi@kali)~$
```

Image 7

7. Findings:

1. Information Disclosure

- **Flag 1 Discovery Description**

During the post-exploitation phase of the assessment, access to the **WordPress administrative dashboard** was successfully obtained. After logging into the WordPress admin panel (/wp-admin), further inspection of the dashboard elements was performed to identify any hidden information or sensitive data that could indicate the presence of a flag.

While reviewing the **Quick Draft** section within the WordPress dashboard, a draft post titled “**flag_1**” was discovered. The content of this draft contained a hash-like string, which represented the first flag of the challenge. This indicated that sensitive information had been improperly stored within the application’s administrative content area without proper protection or restriction.

The presence of this flag within the draft post demonstrates a **sensitive information disclosure issue**, where critical data is accessible to any user who gains administrative access to the WordPress dashboard. This finding highlights the importance of securing administrative credentials and ensuring that sensitive data is not stored within publicly accessible application components.

The discovery of this flag confirms that the attacker was able to successfully gain administrative access to the application and enumerate internal content within the WordPress environment.

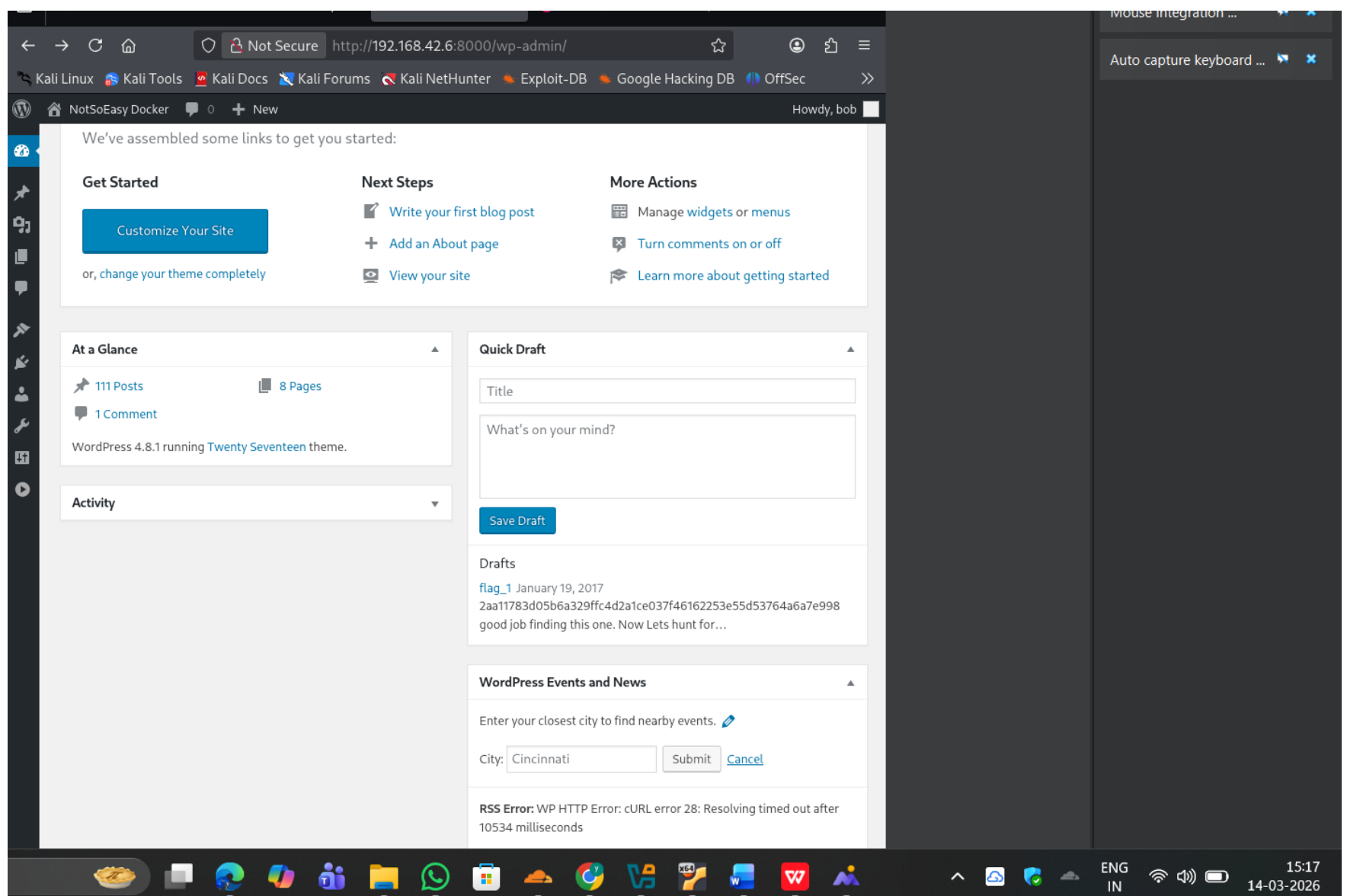


Image 8

Flag 1 String: “ 2aa11783d05b6a329ffc4d2a1ce037f46162253e55d53764a6a7e998”

Impact :

The impact of finding sensitive information—like flags, notes, or drafts—in a publicly accessible or poorly secured area of a CMS (WordPress) is significant:

- **Credential Leakage:** Drafts often contain temporary passwords, API keys, or internal staging URLs that can lead to a full system compromise.
- **Unauthorized Data Access:** Storing sensitive "flags" or notes in Drafts assumes they are private, but any account compromise (even a low-level one) exposes this data.
- **Reconnaissance Goldmine:** Information in robots.txt or Drafts often reveals the directory structure or internal naming conventions of the server
- **Security Posture Assessment:** It signals to an attacker that the administrator is "careless," suggesting that more critical vulnerabilities (like the ones we found later) likely exist.

Remediation:

- **Enforce Account Masking:** Disable Author ID archives by adding a code snippet to functions.php to prevent numerical enumeration (e.g., `/?author=1`).
- **Strict Draft Management:** Implement a "Clean Desk" policy for digital content; delete all temporary or sensitive drafts once they are no longer needed.
- **Hardened Robots.txt:** Use robots.txt only for SEO instructions, never to "hide" sensitive directories. Use server-side access controls (like .htaccess) for real security.
- **Principle of Least Privilege:** Ensure that low-level users cannot view posts or drafts created by administrative accounts.

2. OS Command Injection:

8. Conclusion:

The penetration testing conducted on the **NotSoEasy Docker** environment successfully demonstrated the critical risks associated with outdated software and insecure container configurations. By following the standard **PTES (Penetration Testing Execution Standard)** framework, this project moved from initial network reconnaissance to the total compromise of the host system.

The primary findings reveal that even a well-isolated containerized application can be used as a pivot point for a host-level takeover if security best practices are ignored. The chain of exploitation—starting with **Information Disclosure** (Flag 1), moving to **OS Command Injection** (Flag 2), and concluding with a **Docker Socket Escape** (Flag 3)—highlights the necessity of the "Defense in Depth" strategy.

Ultimately, this assessment proves that security is not a single product but a continuous process. For organizations utilizing containerization, it is imperative to implement strict resource isolation, maintain rigorous patch management for CMS platforms like WordPress, and enforce the principle of least privilege to prevent local privilege escalation.